Post-training Gemma 4 into an offline software-engineering agent on a free T4

Paper Track submission — final. Headline numbers are from the completed held-out run (kernel `ashuraasura/gemma-4-eval-holdout-architecture-2-6`, 11 h 19 m on one T4). The result is negative and is reported as measured.

Abstract

We post-train Gemma 4 E4B into a repository-editing agent that runs entirely offline inside a CPU-sandboxed tool loop. The pipeline is QLoRA supervised fine-tuning (SFT) on 515 compacted, test-verified tool-use trajectories (from 870 built), followed by a GRPO attempt on a disjoint 267-task training pool carved from 291 verified RL tasks (a 24-task dev slice is removed before training), with a test-execution reward. All training and evaluation run on a single free-tier Kaggle T4 (16 GiB). On a frozen, repo-stratified 24-task subset of a 291-task held-out split, both the adapter and the zero-shot base resolve **0 of the 20 tasks whose environments build** (0.0%, 95% Wilson 0.0–16.1%); with no discordant tasks the exact McNemar test is degenerate ($p = 1.0$). SFT changes *how* the agent fails without changing the rate: the adapter keeps the tool protocol but collapses into repeating one rejected action (18 of 19 episodes with at least four calls use at most three distinct calls) and never submits, while the base model emits unparseable text in 6 of 20 episodes and exhausts the 16 GiB card in 12 of 20.

1. Introduction

Software-engineering agents must do three things at once: read an unfamiliar repository, edit it correctly, and verify the edit without network access. Frontier results rely on large models and large-scale execution feedback. We ask what is achievable with a 4B-parameter model, 4-bit quantization, one consumer GPU, and a hard wall-clock budget.

Our contributions are:

1. An end-to-end, reproducible pipeline (data construction → QLoRA SFT → GRPO → held-out evaluation) that fits in free-tier Kaggle sessions.
2. A memory-bounded agent runtime: host-RAM offload of Gemma 4 E4B's 5.25 GiB per-layer embedding table plus a bounded generation context, which turns the long eval episodes that OOMed on a T4 into runs that complete. The offload is E4B-specific; sliced/chunked cross-entropy is the part that generalizes to other sizes.
3. An honest, compute-bounded evaluation on a frozen stratified subset, with sandbox-isolation caveats reported alongside every number and a null result reported as a null result.

2. Related work

- **SWE-bench / SWE-bench Verified** established repository-level issue resolution with hidden tests as the metric.
- **SWE-agent** showed the importance of a small, fixed tool interface (view/edit/search/run) for agent-computer interaction.
- **QLoRA** makes 4-bit fine-tuning of large models feasible on a single GPU.
- **GRPO** removes the value network by normalizing rewards within a group of rollouts for the same prompt, which suits a tight compute budget.

Our work is an independent, small-scale reproduction of this recipe under explicit hardware constraints, not a new algorithm.

3. Method

3.1 Data and splits

- 9 Python repositories are prepared as local checkouts.
- Runnable task pool: 1,786 issue–test pairs whose environments build.
- SFT: 870 trajectories were built and kept only if the final patch passes the task's tests in the sandbox. The trajectories were then compacted to the tool outputs the current agent actually sees. **515 survived the ≤16k-character (~4k-token) budget; the other 355 were excluded, not silently truncated.** The adapter therefore trained on 515 trajectories. Prompts and observations are masked; loss is computed on the model's own reasoning and tool-call tokens only.
- RL: 291 verified tasks; a deterministic 24-task dev slice is removed from training, leaving 267 training tasks, disjoint by task id from SFT and the holdout; the policy sees the issue and repository but never the gold patch. The pools are **not** fully disjoint by code state: 18 RL tasks share a base commit with SFT tasks and 4 share one with holdout tasks (SWE-bench is public). We disclose this rather than claiming a repo-disjoint split.
- Evaluation: a frozen 291-task holdout, untouched by SFT and RL at the task-id level. Because a full 291 × 2-policy evaluation costs ~100+ GPU-hours at the measured T4 speed, the headline evaluation uses a deterministic 24-task subset: holdout rows are bucketed by repository and sampled round-robin (4 django, 4 xarray, 4 pylint, 3 pytest, 3 sphinx, 3 sympy, 2 flask, 1 requests). **Before sampling, we exclude the 23 holdout tasks that share a base commit or issue text with either training pool** (M5 in `AUDIT.md`), so the reported subset is clean at the commit/issue level. The subset file is committed at `data/holdout_eval_subset.jsonl`. Public SWE-bench pre-training exposure cannot be excluded and is stated as a limitation.

3.2 Agent runtime and sandbox

The policy emits exactly one tool call per step from a fixed interface: `read_file`, `search_repo`, `edit_file`, `run_tests`, `submit`. There is no shell: argv is executed directly, binaries are allowlisted, and repository paths are scope-checked. Each test run has CPU-time, memory and process limits and returns a typed result (`pass`, `fail`, `timeout`, `error`).

On Kaggle, `unshare` is blocked by seccomp, so filesystem/network isolation degrades to `best_effort`; every reported number carries that caveat.

3.3 QLoRA SFT

Base model: Gemma 4 E4B (the 12B variant exceeds 16 GiB once full-vocabulary logits, activations and optimizer state are added). The base is loaded in 4-bit NF4 with double quantization; LoRA rank 16 / alpha 32 is applied to all attention and MLP projections. Training uses one epoch, batch size 1 with 16 gradient-accumulation steps, LR 2e-4 with cosine decay, max sequence length 4096, and gradient checkpointing.

Two engineering changes were required to fit the T4:

- **PLE offload (E4B-specific).** Gemma 4 E4B's `embed_tokens_per_layer` table is `[262144, 10752]` bfloat16 ≈ 5.25 GiB and is never trained. It is moved to host RAM; the forward pass gathers on CPU and copies only the `[B, S, 10752]` result to the GPU. This recovers roughly a third of the card. The helper is a no-op on models without that module, so we do **not** claim it transfers to other Gemma 4 sizes.
- **Chunked sliced cross-entropy (general).** The LM head is applied only to supervised positions in chunks of 128, avoiding a full-vocabulary logit tensor. This is the memory fix that carries over to larger

models.

Final SFT loss fell from 0.648 to 0.301 to 0.264 over the logged run (adapter: `ashuraasura/sft-adapter`).

3.4 GRPO

The SFT adapter initializes both the trainable policy and a frozen reference (they share one 4-bit base). For each task we sample `k=4` rollouts through the same agent loop and score each with the sandbox:

```
reward = 1.0 * tests_pass
       + 0.15 * patch_valid
       - 0.005 * steps_taken
       - 0.1 * (no submission)
```

Advantages are group-centered within a task (no std scaling, following the Dr. GRPO correction), validity/step shaping is gated on at least one rollout resolving the task, and a KL penalty (coefficient 0.01) anchors the policy to the frozen SFT adapter. Budget after measurement on the T4: `k=4`, one task per update, 8 steps per episode, 384 new tokens per step, 20 updates, checkpoint every 5 updates, 8-hour wall clock.

4. Experimental setup

Item	Value
Hardware	1× Kaggle T4, 16 GiB, bfloat16 unsupported → fp16 compute
Model	Gemma 4 E4B, 4-bit NF4 + LoRA
Adapter base	`/kaggle/input/models/google/gemma-4/transformers/gemma-4-e4b-it/1` (written in `adapter_config.json`)
SFT data	515 compacted trajectories (from 870 built; 355 over length budget)
RL pool	291 verified tasks; 24-task dev slice, 267 for training
Eval	24-task frozen, repo-stratified subset; 12 steps; 512 new tokens
Eval sample	24 attempted → 4 env failures → 20 scored → 8 clean pairs (no OOM either arm)
Metrics	resolved rate, patch validity, steps-to-submit, failure category
Sandbox	`best_effort` isolation on Kaggle (seccomp blocks `unshare`)

Decoding for evaluation is greedy (temperature 0) so each task has one fixed outcome for the paired McNemar test. A task counts as resolved only if the submitted patch applies to a fresh checkout, touches no test/harness file, and makes the task's tests pass with at least one test executed and none skipped. This blocks the obvious reward hack of editing the tests.

5. Results

Table 1. Held-out results on the frozen 24-task subset (12 steps, 512 new tokens, greedy decoding). "Scored" excludes tasks whose virtualenv could not be built; both arms see exactly the same tasks.

Policy	Attempted	Env. failure	CUDA OOM	Scored	Resolved (95% Wilson)	Patches submitted
Base Gemma 4 E4B, zero-shot	24	4	12	20	0/20 = 0.0% (0.0–16.1%)	0

Policy	Attempted	Env. failure	CUDA OOM	Scored	Resolved (95% Wilson)	Patches submitted
+ QLoRA SFT	24	4	0	20	0/20 = 0.0% (0.0–16.1%)	0
+ GRPO	not evaluated — no learning signal, no checkpoint	—	—	—	—	—

The run is a null result. Neither policy resolved a task, so there is no patch-validity or steps-to-submit statistic to report, and the paired comparison contains no discordant tasks (treatment-only 0, baseline-only 0, exact McNemar p = 1.0). We state this as measured rather than reframing it.

Twelve of the twenty base episodes ended in CUDA OOM, so the 20-task rate is not a clean capability estimate for the base policy. Restricting to the eight pairs where neither arm hit an infrastructure failure gives 0/8 = 0.0% (95% Wilson 0.0–32.4%) in both arms, again with zero discordant pairs. The OOM is a property of the base policy's context behaviour (§5.2), not of an asymmetric harness: both arms use the same 4-bit loader, the same 12k-token prompt cap and the same per-step cache release.

Per repository, on the 20 scored pairs, both arms are 0/4 django, 0/4 pylint, 0/3 pytest, 0/3 sphinx, 0/3 sympy, 0/2 flask and 0/1 requests.

5.1 Why the adapter never submits

Table 2. Tool use over the 20 scored episodes per arm.

Signal	QLoRA SFT	Base
tool calls	229	123
`read_file`, ranged / whole-file	119 / 22	0 / 48
`edit_file`	79	2
`search_repo`	2	70
`run_tests`	7	2
`submit`	0	1 (empty arguments)
episodes with ≥4 calls using ≤3 distinct calls	18/19	2/16
episodes that never submit	20/20	19/20
median completion tokens per call	36	92

The adapter mostly holds the tool protocol — all 20 scored episodes emit parseable tool calls and only one ends on an unparseable generation — but it is stuck. Eleven episodes hit `error: path does not exist` as their first tool error (for example `pytest/_matchers.py` instead of `src/_pytest/_matchers.py`), five hit a hunk whose context does not match the file, two hit a malformed hunk (`@@ -222,10 +222,12` followed by 8 old and 10 new lines), and two produced no tool error at all. In 18 of the 19 episodes that made at least four calls, the policy used at most three distinct calls across twelve steps; most used one or two. It re-issues the rejected action verbatim: `django-10390` repeats a malformed hunk 11 times, `pytest-10081` repeats the same non-existent-path read 11 times, and `requests-5087` alternates one edit with `run_tests` six times while the tests keep failing. Completion lengths are 36 tokens at the median, so the 512-token cap is not what stops it; one episode ends on a diff that stops mid-file and cannot be parsed.

The demonstrations explain the shape of this failure. All 515 training trajectories call `submit` immediately after a `run_tests` result of `status=pass` (515/515), so the policy learned a success-conditioned transition into `submit` and has no branch that reaches `submit` from a failing state. The training set also contains exactly one

class of tool error — `patch does not apply` — in 125 observations across 125 trajectories, and no example of a path-resolution error, which is the most common eval failure (11/20). Supervision alone does not explain the loop, however: five episodes fail with the same `patch does not apply` class that the data does contain, and the adapter still repeats it 10–12 times instead of revising. Imitating a successful trajectory does not by itself teach recovery.

5.2 Base arm: context growth, then OOM

The base arm fails differently. It never uses line ranges (0 of 48 reads; the adapter uses ranges in 119 of 141), calls `search_repo` 70 times, and writes longer completions (median 92 tokens). Its prompts therefore grow much faster and run into the 12k-token cap, and 12 of 20 episodes died with `CUDA out of memory` between steps 3 and 11. Of the remaining eight, six emitted prose instead of a tool call (`no tool call found in model output`) and two exhausted the step budget. Its single `submit` carried empty arguments.

We do not conclude that the base model is incapable: 12 of 20 episodes were truncated by memory before it could submit. The comparison we can defend is the eight clean pairs, where both arms resolve 0/8.

5.3 GRPO attempt (negative result)

We ran GRPO from the SFT adapter with `k=4`, one task per update, 8 steps, 384 new tokens, and 20 planned updates. Only **4 updates completed** before the run was stopped. Loss, policy-gradient norm, and KL were exactly `0.0000` on every update, and the mean group reward stayed flat: `-0.136`, `-0.134`, `-0.140`, `-0.139` (best `-0.130`, `-0.120`, `-0.140`, `-0.135`). All four rollouts per task failed, so the group-centered advantages were zero and no gradient was applied; update 1 also produced an unparseable tool call (`no tool call found in model output`). Checkpoints are written every 5 updates, so **no RL adapter exists**. The evaluated policy is therefore the SFT adapter; the GRPO attempt is reported as a negative result, not as an improvement.

SFT loss curve (logged): 0.648 → 0.301 → 0.264.

6. Ablations

We report only ablations the compute budget actually allows; anything not run is listed as not run rather than implied.

- **SFT vs zero-shot base** — run on the same 24 tasks with the same harness. Both arms resolve 0/20 scored tasks; there are 0 discordant pairs and the exact McNemar p is 1.0. This is a null result, not evidence of equivalence: at this sample size the design can only detect a large difference, and the base arm additionally lost 12/20 episodes to CUDA OOM.
- **SFT vs SFT+GRPO** — not run: GRPO produced zero gradient for 4 updates and no adapter was checkpointed. The SFT adapter is the final policy; this is a negative result, not an omitted ablation.
- **Group size k** — RL v10 used k=2 and produced zero gradient because both rollouts failed (advantages ≈ ±0.0025); the final run uses k=4. This is a measured debugging result, not a clean ablation.
- **Reward shaping** — not ablated; shaping is gated and small by design.
- **Model size (12B vs E4B)** — not a choice: 12B OOMs on a T4. Reported as a constraint, not an ablation.

7. Failure modes

Categories are assigned from the rollout record, the test log, and the gold patch (for file overlap). Counts are over the 24 attempted tasks per arm:

Category	SFT	Base	Definition
`step_budget_exhaustion`	19	2	no `submit` within 12 steps

Category	SFT	Base	Definition
`policy_error`	1	6	episode ended with output containing no parseable tool call
`infrastructure_error`	0	12	episode killed by CUDA OOM before submission
`test_environment_issue`	4	4	virtualenv build failed before inference
`invalid_patch` / `wrong_file` / `incomplete_patch`	0	0	never reached — no patch was ever submitted

The last row is not an oversight: because neither arm submitted a patch, the patch-level taxonomy that the harness implements has no observations in this run. We report it empty rather than assigning categories to episodes that never produced a diff.

Eval v4 (pre-fix) was diagnostic: the SFT adapter OOMed after 14–21 steps because the 5.25 GiB PLE table stayed resident while the KV cache grew. The fix (PLE host offload, 12k-token head+tail prompt cap, per-step cache release) was smoke-tested in eval v5 and removed SFT-arm OOM entirely in v6. It did not remove base-arm OOM, which is driven by that policy's whole-file reads rather than by model size (§5.2).

8. Limitations and honest scope

- **Statistical power and a null result.** The full 291-task × 2-policy holdout is ~100+ GPU-hours at measured speed and was not run. Headline numbers are on the committed 24-task stratified subset. Both arms resolved 0 of 20 scored tasks (95% Wilson 0.0–16.1%), so the exact McNemar test has no discordant pairs and returns p = 1.0. This run can rule out a large effect and nothing smaller; we do not claim that SFT and base are equivalent, only that this measurement does not separate them.
- **Base-arm OOM confound.** 12 of 20 base episodes ran out of GPU memory before submitting, because the base policy reads whole files (0/48 ranged reads) and pushes its prompt to the 12k-token cap. Its 0/20 is therefore not a fair capability estimate; the defensible comparison is the eight clean pairs, where both arms are 0/8. A fair base evaluation needs a policy-side context budget, not a larger card.
- **Sandbox isolation.** Kaggle blocks `unshare`; filesystem/network isolation is `best_effort`. A malicious policy could in principle reach the host filesystem. Reward and eval numbers are therefore not fully isolated.
- **Model size.** E4B, not 12B: 12B does not fit a 16 GiB T4 with full-vocabulary logits and activations.
- **SFT data selection.** 355 of 870 built trajectories exceeded the 16k-character compaction budget and were excluded. If long trajectories correlate with harder tasks, the adapter underrepresents them.
- **Supervision coverage.** The 515 retained trajectories contain exactly one tool-error class (`patch does not apply`, 125 observations) and no path-resolution errors, although wrong paths cause 11 of the 20 scored SFT failures. Every trajectory submits only after a passing test run. The adapter was therefore never shown how to recover from the dominant eval failure, and the paper reports that as a data-construction defect rather than a model property. Training on rejected-action/recovery pairs is the obvious next step and was not run.
- **Holdout overlap.** The 291-task holdout contains 23 tasks that share a base commit or issue text with the SFT/RL pools; they are excluded from the headline 24-task subset, so the subset is not a uniform sample of the full holdout. Public SWE-bench pre-training exposure remains possible.
- **Task-environment coverage.** Four xarray tasks in the subset pin `pandas==1.5.3` with `numpy<2`; that stack has no Python 3.13 wheel and cannot be built on the Kaggle image. Both arms fail at environment setup, so these tasks contribute no policy signal and the effective paired sample is 20 tasks. They are reported as environment failures, not as policy failures, and are not silently dropped from the attempted count.

- **Decoding.** The final paired comparison uses greedy decoding (temperature 0) so each task has one fixed outcome for McNemar; sampling variance and seed variance are not measured.
- **Single seed, single run.** No error bars across seeds.
- **Adapter portability.** LoRA weights are tied to E4B's layer shapes and cannot be loaded into a 12B or other-size model. A larger model requires retraining the adapter, not patching this one. We make no claim that a particular larger Gemma 4 variant exists.
- **RL scale.** The GRPO attempt was stopped after 4 of 20 planned updates; it produced no gradient and no checkpoint, so no RL result is reported. This is a negative result about the reward signal at this scale, not an estimate of what a longer RL run would achieve.
- **No teacher-model distillation.** SFT trajectories are replayed from test-passing gold patches and filtered rollouts, not generated by a larger teacher. Any future distillation run must first check the competition rules on outside teacher models.

9. Reproducibility

- Code: this repository (`gemma4-coding-agent`). The eval kernel ran commit `1f2cc9b` with `run_eval` sha `2fb5f2451dd7`; the paper, the analysis script (`eval.final_analysis`) and the reporting fixes are at repository HEAD.
- Numbers: `python -m eval.final_analysis sft.jsonl base.jsonl --sft-traj sft_traj.jsonl --base-traj base_traj.jsonl --sft-data data/sft_trajectories.compact.jsonl` reproduces Tables 1–2 and the supervision counts from the released JSONL.
- Kaggle kernels: `ashuraasura/gemma-4-qlora-sft-architecture-2-4`, `ashuraasura/gemma-4-grpo-rl`, `ashuraasura/gemma-4-eval-holdout-architecture-2-6`.
- Adapter: `ashuraasura/sft-adapter`.
- Evaluation artifacts: per-task JSONL plus trajectories from the eval kernel; summary JSON contains resolved rate, patch validity, steps to submit and per-repo breakdown.
- Deterministic subset: `data/holdout_eval_subset.jsonl`.

Appendix A. Compute accounting

Stage	Measured
SFT	22 kernel versions; PLE offload frees 5.25 GiB; CE chunk 128
Eval (pre-fix)	~50–80 s/step; 19.5 min for a 21-step SFT episode
Final eval, SFT arm	9,801 s (2 h 43 m) for 24 tasks, 0 OOM
Final eval, base arm	remainder of an 11 h 19 m kernel (≈8 h 30 m), 12 OOM
RL rollout	619 s for 2 rollouts × 6 steps + one update (k=2 smoke)
Kaggle quota	24 h 17 m available of 30 h before the final runs; 11 h 19 m used by v6